Identificazione Braccio Robotico

Giulio Sergi, Andrea Sodini, Matilda Ferri

February 2025

1 Modello del Sistema

1.1 Scelta dei Parametri del Sistema

Il sistema proposto in esame è rappresentato da un manipolatore planare RR attuato tramite la coppia τ_{m1} agente sul link incernierato al suolo e una coppia τ_{m2} agente sul giunto rotoidale che unisce il primo al secondo link. I giunti presentano attriti viscosi non trascurabili c_1 e c_2, così come coppie di disturbo pari a $\tau_d = [\tau_{d1}, \tau_{d2}]^T$. Per quanto riguarda i link, essi sono caratterizzati in lunghezza (l_1, l_2), massa (M_1, M_2) e momento di inerzia (I_1, I_2) rispetto al proprio centro di massa (h_1, h_2). Complessivamente, il manipolatore è descritto dalla seguente figura.

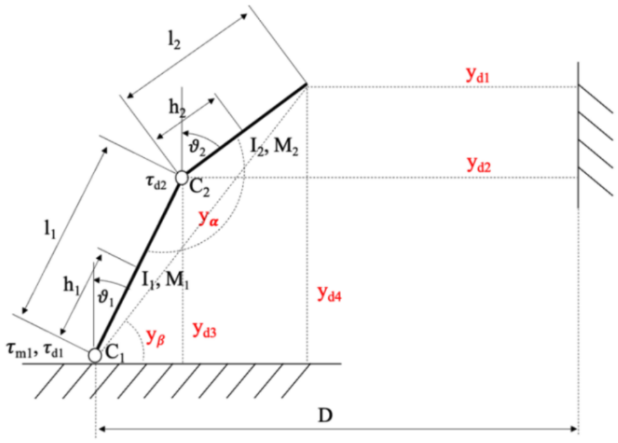


Figure 1: Manipolatore Planare RR

La prima fase di modellizzazione prevede la definizione dei valori da attribuire ai suddetti parametri di modello. Nella casistica studiata, abbiamo fatto riferimento ad un manipolatore seriale con le seguenti caratteristiche:

- **Lunghezze dei link**: $l_1 = 1$ m, $l_2 = 0.5$ m
  Questi valori sono tipici per manipolatori industriali di medie dimensioni, come bracci robotici per operazioni di assemblaggio o manipolazione di oggetti in ambienti controllati. Il secondo link è più corto (l_2) rispetto al primo in modo da ridurre il momento d'inerzia e migliorare la precisione del controllo di attuazione sull'End Effector. Manipolatori come il *KUKA LBR iiwa* presentano strutture e dimensioni di link comparabili, a cui abbiamo fatto riferimento.

- **Masse dei link**: $M_1 = 20$ kg, $M_2 = 10$ kg
  La caratterizzazione delle masse riflette la definizione delle lunghezze dei link, in quanto la prima massa è doppia rispetto alla seconda. In generale, il primo link è tendenzialmente più massiccio in quanto deve sostenere il peso del secondo e garantire una maggiore rigidità strutturale. Questi valori sono anche compatibili con quelli del manipolatore *KUKA LBR iiwa*, in cui i materiali tipici includono alluminio per i link e acciaio per i giunti critici.

- **Momenti d'inerzia**: $I_1 = \frac{1}{3}M_1 l_1^2$, $I_2 = \frac{1}{3}M_2 l_2^2$
  I momenti d'inerzia sono calcolati assumendo che i link siano barre uniformi con rotazione attorno a un'estremità.

- **Distanze del baricentro rispetto all'asse di rotazione**: $h_1 = 0.5$ m, $h_2 = 0.25$ m
  Questi valori corrispondono alla metà delle lunghezze dei link, ipotizzando una distribuzione uniforme della massa.
- **Coefficienti di attrito viscoso**: $c_1 = 0.07$, $c_2 = 0.1$
  L'attrito viscoso nei giunti è un parametro che dipende dai motori e dai riduttori utilizzati. Supponendo l'uso di giunti con trasmissione armonica, come quelli usati nel sopra citato manipolatore seriale industriale, i valori tipici di attrito viscoso variano tra 0.05 e 0.1, compatibili dunque al modello proposto.
- **Accelerazione gravitazionale**: $g = 9.81$ m/s^2
- **Periodo di campionamento**: $dt = 0.001$ s
  Il valore viene utilizzato al fine di discretizzare il modello del sistema e dei sensori in ambiente Simulink.
- **Distanza tra Sensore Ultrasuoni e Giunto al Suolo**: $D = 2$ m
  Rappresenta la distanza tra il sensore ad Ultrasuoni e il giunto incernierato al suolo nella sua proiezione lungo l'asse delle ascisse. Il sensore ad Ultrasuoni ha lo scopo di misurare la distanza dal giunto rotoidale che unisce i due link.
- **Varianza del rumore di processo**: $\sigma_W = 4$
  Questo valore tiene di conto di incertezze di modello e disturbi esterni, in accordo con i segnali di disturbo posti in ingresso al modello reale del sistema in fase di simulazione.
- **Varianze del rumore di misura**: $\sigma_{V_1} = 0.1$, $\sigma_{V_2} = 0.05$
  I parametri di varianza del rumore di misura tengono conto di incertezze sulle misure effettuate, provocate da errori di calibrazione dello strumento e disturbi agenti sui dispositivi.
- **Condizioni Iniziali sullo Stato del Modello**: $\theta_1 = -0.7416$, $\theta_2 = 1.0178$, $\dot{\theta}_1 = 0$, $\dot{\theta}_2 = 0$
  Rappresentano le condizioni iniziali del manipolatore seriale. I valori proposti fanno riferimento alle variabili di stato che il robot deve assumere inizialmente al fine porre l'End Effector nel punto del piano (-0,25; 1) appartenente ad una circonferenza centrata in (0; 1).

1.2 Dinamica del Manipolatore

Il modello dinamico del manipolatore seriale è descritto indicando con $\theta = [\theta_1, \theta_2]^T$ gli angoli che ciascun link forma rispetto alla verticale e con $\tau_m = [\tau_{m1}, \tau_{m2}]^T$ il vettore delle coppie di attuazione. Le equazioni che modellano la dinamica del sistema sono:

$$M(\vartheta)\ddot{\vartheta} + N\dot{\vartheta} + q(\vartheta) = G\tau_m + T\tau_d$$

in cui le matrici dinamiche sono definite come segue:

$$M = \begin{bmatrix} I_1 + M_1 h_1^2 + M_2 l_1^2 & l_1 M_2 h_2 \cos(\theta_1 - \theta_2) \\ l_1 M_2 h_2 \cos(\theta_1 - \theta_2) & I_2 + M_2 h_2^2 \end{bmatrix}$$

$$N = \begin{bmatrix} c_1 + c_2 & -c_2 \\ -c_2 & c_2 \end{bmatrix}$$

$$q = \begin{bmatrix} l_1 M_2 h_2 \sin(\theta_1 - \theta_2)\dot{\theta}_2^2 - (M_1 h_1 + M_2 l_1) g \sin(\theta_1) \\ -l_1 M_2 h_2 \sin(\theta_1 - \theta_2)\dot{\theta}_1^2 - M_2 h_2 g \sin(\theta_2) \end{bmatrix}$$

$$G = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$T = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$$

E' possibile ricavare le accelerazioni angolari $\ddot{\theta}$ utilizzando la seguente formula ottenuta per inversione della dinamica del sistema:

$$\ddot{\theta} = M^{-1}\left(G\begin{bmatrix} T_{m1} \\ T_{m2} \end{bmatrix} + T\begin{bmatrix} T_{d1} \\ T_{d2} \end{bmatrix} - N\begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} - q\right)$$

Infine, gli stati del manipolatore, ossia angoli e velocità angolari, vengono aggiornati nel tempo utilizzando il metodo di Eulero:

$$\theta_1^{\text{next}} = \theta_1 + \dot{\theta}_1 dt$$
$$\theta_2^{\text{next}} = \theta_2 + \dot{\theta}_2 dt$$
$$\dot{\theta}_1^{\text{next}} = \dot{\theta}_1 + \ddot{\theta}_1 dt$$
$$\dot{\theta}_2^{\text{next}} = \dot{\theta}_2 + \ddot{\theta}_2 dt$$

Lo stato successivo del manipolatore è quindi dato dal vettore:

$$x_{\text{next}} = \begin{bmatrix} \theta_1^{\text{next}} \\ \theta_2^{\text{next}} \\ \dot{\theta}_1^{\text{next}} \\ \dot{\theta}_2^{\text{next}} \end{bmatrix}$$

Complessivamente, il modello del sistema è stato implementato in ambiente Simulink mediante un blocco MATLAB Function, i cui stati sono opportunamente retroazionati sfruttando l'utilizzo di un blocco Time-Delay, utile per la corretta evoluzione del sistema tempo discreto. Il modello implementato è visibile in figura 2.

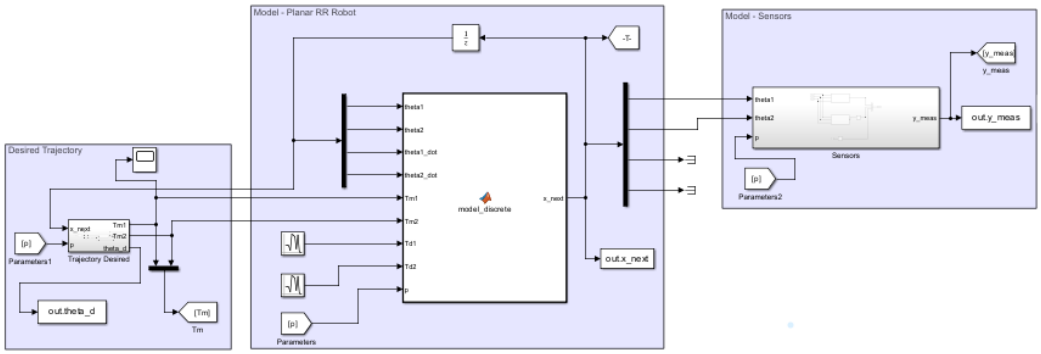


Figure 2: Modello complessivo del Manipolatore Planare RR

1.3 Validazione del Modello

Al fine di validare il modello del manipolatore seriale proposto, è stato scelto un approccio di inseguimento di una traiettoria desiderata. L'implementazione si basa su tre componenti principali: la generazione della traiettoria desiderata, la cinematica inversa per calcolare lo stato del sistema a partire da posizioni sul piano cartesiano e il controllo PD per il tracking fluido della traiettoria.

Nella prima fase, il codice `trajectory` genera una generica traiettoria ellittica per il sistema. La traiettoria è definita dai parametri del semi-asse maggiore a e del semi-asse minore b, insieme alla posizione del centro (X_c, Y_c) e alla velocità angolare ω. La posizione desiderata lungo gli assi X e Y è calcolata come:

$$X_d = X_c + a\cos(\omega t), \quad Y_d = Y_c + b\sin(\omega t) \tag{1}$$

Queste equazioni possono essere facilmente modificate per descrivere una traiettoria circolare, definendo i parametri a e b coincidenti. Nel caso in esame i parametri della traiettoria circolare desiderata su cui è stato validato il modello risultano essere:

$$a = b = 0.25$$
$$center = [0; 1]$$
$$\omega = -0.5$$

Il codice `inverse_kinematics` è utile nella seconda fase al fine di risolvere il problema della cinematica inversa per il manipolatore RR, determinando gli angoli θ_1 e θ_2 e le velocità angolari $\dot{\theta}_1$

e $\dot{\theta}_2$ necessari per raggiungere la posizione desiderata (x_d, y_d). Viene effettuata inoltre una verifica della raggiungibilità del punto desiderato confrontando la distanza tra l'origine e il punto target con la lunghezza massima del braccio robotico.

Infine, per la terza fase, il codice `pd_control` implementa un controllo proporzionale-derivativo (PD) per garantire il tracking della traiettoria desiderata. Il controllo è basato sugli errori di posizione e e velocità $\dot{e}$, definiti come:

$$e = \theta_d - \theta, \quad \dot{e} = \dot{\theta}_d - \dot{\theta} \tag{2}$$

Il controllo PD ci fornisce dunque le coppie ai giunti desiderate secondo la formula:

$$\tau = k_p e + k_d \dot{e} + G \tag{3}$$

dove G rappresenta il contributo della gravità.

Le uscite in coppia del blocco di generazione della traiettoria desiderata sono determinanti in fase attuazione del modello, poichè forniscono il controllo adatto all'inseguimento della specifica traiettoria. Nella figura seguente si ripropongono le fasi appena descritte nella loro implementazione in ambiente simulativo.

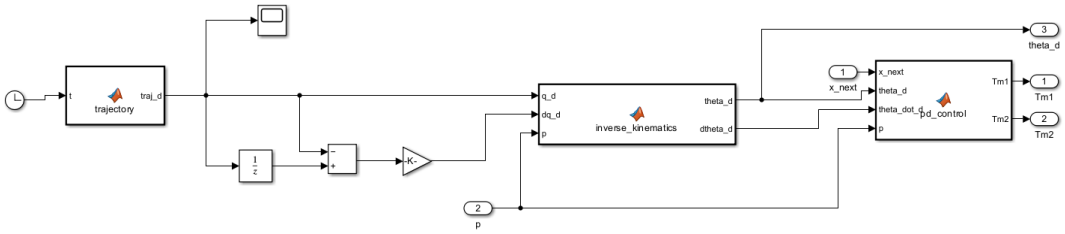


Figure 3: Implementazione inseguimento della Traiettoria desiderata

2 Modello dei Sensori

Il modello in esame prevede la misura di due grandezze relative all'angolo y_α formato tra il primo e il secondo link e la distanza y_{d2} tra il sensore stesso e il giunto rotoidale tra i due link. I modelli di misura relativi ai due sensori proposti fanno riferimento ad un Encoder Ottico per la misura angolare e ad un Sensore ad Ultrasuoni per la misura della distanza, strumenti di comune impiego nell'ambito dell'automazione industriale. I sensori sono stati modellati in maniera tale da operare a due frequenze di campionamento diverse: l'Encoder Ottico produce un segnale in uscita coerentemente con il periodo di campionamento del modello reale (0.001s), mentre il Sensore ad Ultrasuoni genera un nuovo campione di misura ogni 0.003s. Questo aspetto verrà valutato in dettaglio in fase di correzione in stima dinamica.

2.1 Encoder Ottico

L'Encoder Ottico è un dispositivo che converte la rotazione meccanica in segnali elettrici codificati, permettendo di effettuare indirettamente una misura sulla posizione angolare di un giunto, ed eventualmente anche sulla sua velocità angolare. Un Encoder Ottico risulta particolamente adatto per applicazioni robotiche, come quella in esame, in quanto presenta una ottima affidabilità ed una elevata risoluzione. Al fine di definire il seguente modello sono stati presi come riferimento gli Encoder Ottici Incrementali della serie Omron E6B2, con ampio impiego nei settori robotici e industriali.

Dapprima, il modello prevede la misura dell'angolo misurato ang_{mis} in base alla differenza tra due angoli θ_1 e θ_2, tenendo conto che entrambi gli angoli sono misurati rispetto ad un riferimento verticale. La formula per calcolare ang_{mis} dipende dal segno di θ_1, ossia in base alla sua posizione relativa rispetto alla verticale di riferimento, e viene determinata come segue:

$$ang_{\text{mis}} = \begin{cases} \pi - \theta_1 + \theta_2 & \text{se } \theta_1 > 0 \\ \pi + \theta_1 - \theta_2 & \text{se } \theta_1 \leq 0 \end{cases}$$

Dove θ_1 e θ_2 sono gli angoli che vengono prelevati direttamente dall'evoluzione del modello del manipolatore, in cui si tiene conto della geometria relativa tra i due angoli, come in figura 1.

Viene generato un valore casuale v_{encoder} che rappresenta il rumore associato alla misura dell'encoder, generato secondo una distribuzione normale con media 0 e deviazione standard σ_{encoder}, come segue:

$$v_{\text{encoder}} = \sigma_{\text{encoder}} \cdot \text{randn}$$

Per introdurre la possibilità di generare misurazioni spurie (outlier), viene inoltre generata una variabile booleana $it_{\text{is_outlier}}$ che simula la presenza di errori grossolani nei dati. La variabile è definita come:

$$it_is_outlier = \begin{cases} 1 & \text{con probabilità del } 1\% \\ 0 & \text{altrimenti} \end{cases}$$

Infine, il modello di misura dell'angolo y_α, che rappresenta la misura complessiva dell'angolo tenendo conto sia del rumore che degli outlier, è dato dalla seguente espressione:

$$y_\alpha = ang_{\text{mis}} + v_{\text{encoder}} + 50 \cdot (\text{rand}(1,1) - 0.5) \cdot it_is_outlier \tag{4}$$

Nel caso in cui si verifichi un outlier ($it_is_outlier = 1$), viene aggiunto un disturbo modellato con una densità di probabilità uniforme che commisura errori occasionali da parte della strumentazione, derivati da interferenze ambientali, variazioni termiche o generici errori di lettura spuri da parte dell'Encoder. Come mostra la figura 4, l'uscita dell'encoder ottico presenta una forma compatibile con la misura perturbata dell'angolo y_α a cui si aggiungono degli eventi occasionali spurii in corrispondenza di solo alcuni istanti temporali di campionamento.

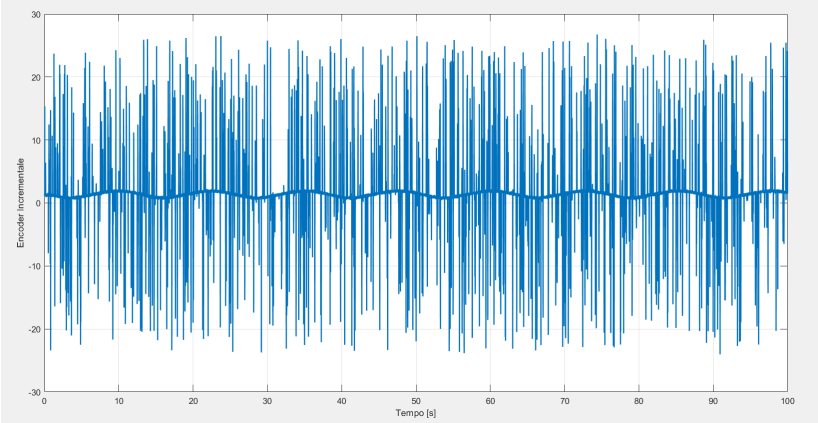


Figure 4: Misura Perturbata dell'angolo y_α con Outliers

2.2 Sensore ad Ultrasuoni

Per quanto riguarda la stima della distanza, si fa uso di un Sensore ad Ultrasuoni ispirato a quelli della serie Pepperl+Fuchs. Analogamente alla casistica dell'Encoder, anche in questo caso il modello implementato ha lo scopo di simulare una misura proveniente da un sensore ad ultrasuoni, tenendo conto di rumore di misurazione e possibili outlier.

Nel corpo della funzione, viene prima calcolata la distanza misurata dal sensore x, in funzione della posizione dell'angolo θ_1 rispetto alla verticale di riferimento:

$$x = \begin{cases} D - l_1 \sin(\theta_1) & \text{se } \theta_1 > 0 \\ D + l_1 \sin(\theta_1) & \text{se } \theta_1 \leq 0 \end{cases}$$

La generazione del rumore di misurazione tramite campionamento di una distribuzione normale con varianza (σ_{sensUltr}), così come la definizione degli eventi di outliers con probabilità dell'1%, sono implementati in maniera del tutto analoga alla casistica dell'Encoder Incrementale.

La misura finale del sensore d_{mis} viene restituita come la somma della distanza calcolata x, del rumore di misurazione e dell'eventuale outlier, come riportato nella seguente equazione:

$$y_{d2} = x + \sigma_{\text{sensor}} \cdot \text{randn} + 50 \cdot (\text{rand} - 0.5) \cdot \texttt{it_is_outlier2}$$

In questo caso, gli outlier possono derivare da riflessioni multiple dell'impulso sonoro, superfici non ottimali per la riflessione del segnale o variazioni ambientali repentine che influenzano la velocità del suono nell'aria. Complessivamente, il segnale y_{d2} generato in uscita dal modello del Sensore ad Ultrasuoni, unito ad eventi spuri occasionali, è rappresentato in figura 5.

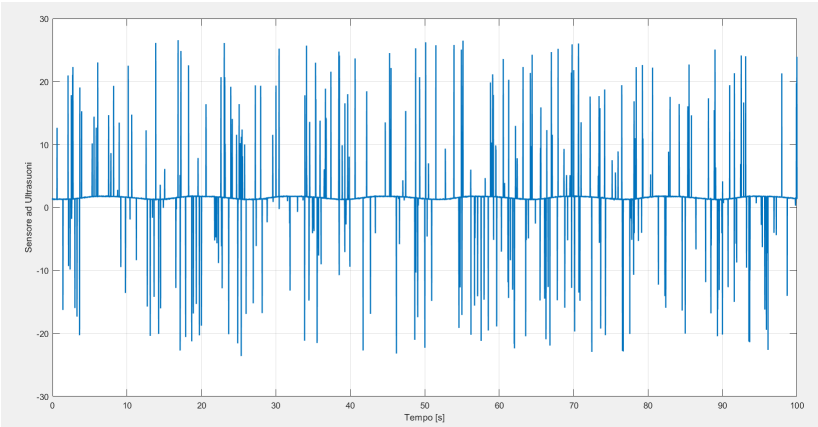


Figure 5: Misura Perturbata della distanza y_{d2} con Outliers

3 Stima UKF

Il primo metodo usato per stimare il movimento del braccio è la stima UKF, implementata utilizzando due blocchi principali :

- **Predizione**
- **Correzione**

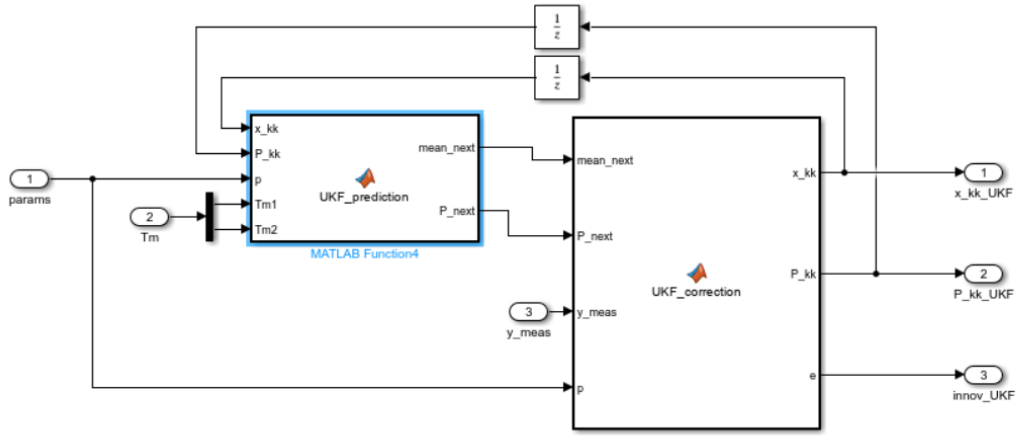


Figure 6: Predizione e Correzione UKF

Nei blocchi di ritardo è contenuta l'inizializzazione per il primo passo uguale alla condizioni iniziali impostate per il braccio reale.

3.1 Predizione

Quasto blocco prende in ingresso lo stato $x_{k|k}$ e la varianza $P_{k|k}$ in uscita dalla correzione e li propaga in $x_{k+1|k}$ (mean_next) e $P_{k+1|k}$ (P_next). Implementando il metodo Unscented la parte fondamentale della predizione è la generazione dei sigma-point creati partendo dalla scelta dei parametri **Alpha=1** (non scalata), **Beta=2**, **Kappa=0**.
Si procede alla generazione dei pesi, diagonalizzazione e creazione dei punti simmetrici come da teoria (Figura7).
Dopo aver creato i sigma-point si inseriscono nella dinamica del sistema braccio visto in precedenza creando i sigma point propogati (sigma_points_next).
Si conclude la predizione calcolando:

- la media di tutti i sigma_point_next, che ci fornirà $x_{k+1|k}$ da fornire alla correzione
- tilde_next, ovvero l'errore tra la media appena calcolata e tutti i sigma_point_next che ci sevirà per implementare la seguente formula:

$$P_{\text{next}} = w_c(i)\left(\tilde{\mathbf{x}}_{\text{next}}(:,i)\tilde{\mathbf{x}}_{\text{next}}(:,i)^T\right)$$

3.2 Correzione

Questo blocco invece prende come ingressi $x_{k+1|k}$ e $P_{k+1|k}$ dalla predizione e implementa il filtraggio utilizzando le misure dei sensori in tempo reale, anch'esse in ingresso (y_meas).
Come nel blocco precedente, anche in questo caso generiamo dei sigma-point allo stesso modo, ma a differenza di prima li andiamo a sostituire nelle dinamiche dei modelli dei sensori in modo da ottenere delle y_cor derivanti dai sigma-point.
Anche in questo caso procediamo ad effettuare una media delle y_cor e delle y_tilde_next identicamente a prima.
Concludiamo la correzione calcolandoci **innovazione(ek)**, **covarianza innovazione(Sk)**, **guadagno correzione(Lk)** e facendo un controllo sulla **distanza di Mahalanobis**. Quest'ultima confronta la

suddetta distanza con una soglia da noi definita mahalanobis_threshold=4 e ci permette di identificare eventuali outliers dei sensori, non eseguendo la corrente correzione. Si fa presente che per semplicità abbiamo scelto di non fare un doppio controllo sui singoli sensori e di scartare entrambe le misure in caso di identificazione di outliers anche se generato da uno solo dei due sensori. Questa scelta non ha comunque inciso negativamente sui risultati grazie all'alto numero di misure che si effettuano (una ogni 0.001s).

```
lambda = alpha^2*(enne + kappa) - enne;

% calcolo pesi
wc = zeros(1, 2 * enne + 1);
w0 = lambda/(lambda + enne);
wc(1) = w0 + 1 - alpha^2 + beta;      %secondo ordine
wc(2:2*enne+1) = 1/2/(enne+lambda);

wm = wc;                              %primo ordine
wm(1) = w0;

% SVD
[U,S,~] = svd(P_kk);
GAMMA = U*sqrt(S);

% generazione 2n1 sigma-point
sigma_points = zeros(enne, 2*enne+1);
sigma_points(:,1) = x_kk;
for i = 1:size(GAMMA,2)
    sigma_points(:,i+1)      =  x_kk + sqrt(enne+lambda)*GAMMA(:,i);
    sigma_points(:,i+1+enne) =  x_kk - sqrt(enne+lambda)*GAMMA(:,i);
end
```

Figure 7: Creazione pesi e sigma-point

```
S_k = zeros(2,2);
cross_cov = zeros(enne,2);
for i = 1:size(sigma_points,2)
    S_k = S_k + wc(i)*ytilde_next(:,i)*ytilde_next(:,i)';
    cross_cov = cross_cov + wc(i) * (xtilde_next(:,i) * ytilde_next(:,i)');
end

e = y_meas - ymean_next;
e(1,1) = atan2(sin(e(1,1)),cos(e(1,1)));
S_k = S_k + R;
L_k = cross_cov * pinv(S_k);
mahalanobis_distance = sqrt(e' * pinv(S_k) * e);

if mahalanobis_distance>mahalanobis_threshold && MAHALANOBIS
    x_kk = mean_next;
    P_kk = P_next;
    identified_outlier = 1;
else
    x_kk = mean_next + L_k * (e);
    P_kk = P_next - L_k * S_k * L_k';
end
```

Figure 8: Calcoli finali Correzione

3.3 Risultati

Per verificare la qualità della stima effettuata sono stati fatti vari grafici e confronti:

- **Stima UKF**

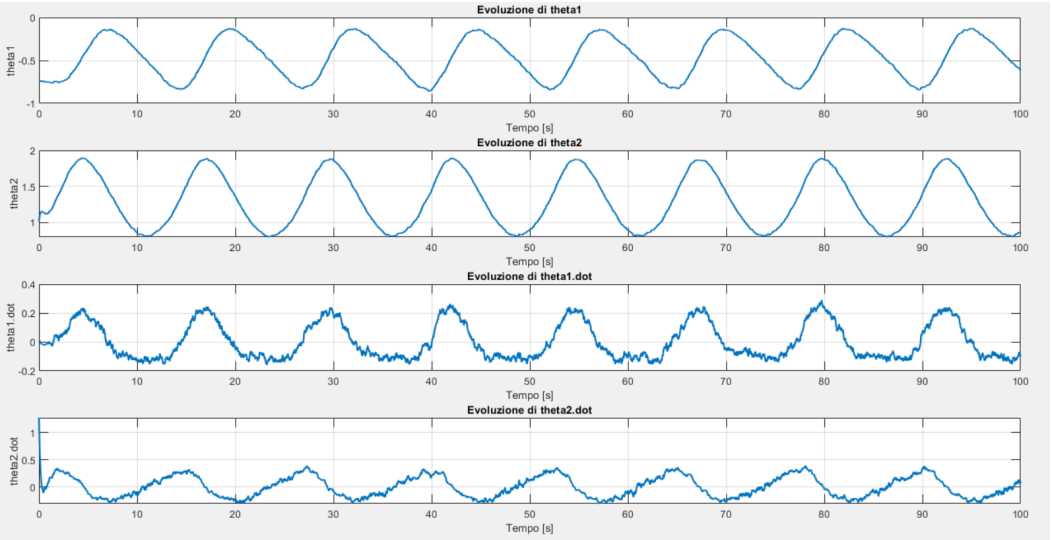


Figure 9: Stati stimati con UKF

Come possiamo notare la stima UKF per quanto riguarda gli angoli ha un tratto abbastanza regolare, cosa che invece peggiora nelle velocità angolari. Questo effetto tremolante è dato su tutti gli stati dal rumore associato alle misure, e quindi dalla varianza che abbiamo associato ad esso. Le velocità angolari sono quelle che, durante la fase di predizione, soffrono maggiormente di errori di stima a causa della dinamica stessa e quindi vengono maggiormente corrette dalle misure dei sensori in fase di correzione.
Questo risultato è osservabile sia con o senza la generazione di outlier, questo ci indica che la distanza di Mahalanobis con la soglia scelta riconoscono ed eliminano in maniera efficiente gli outlier.

- **Confronto modello e UKF**

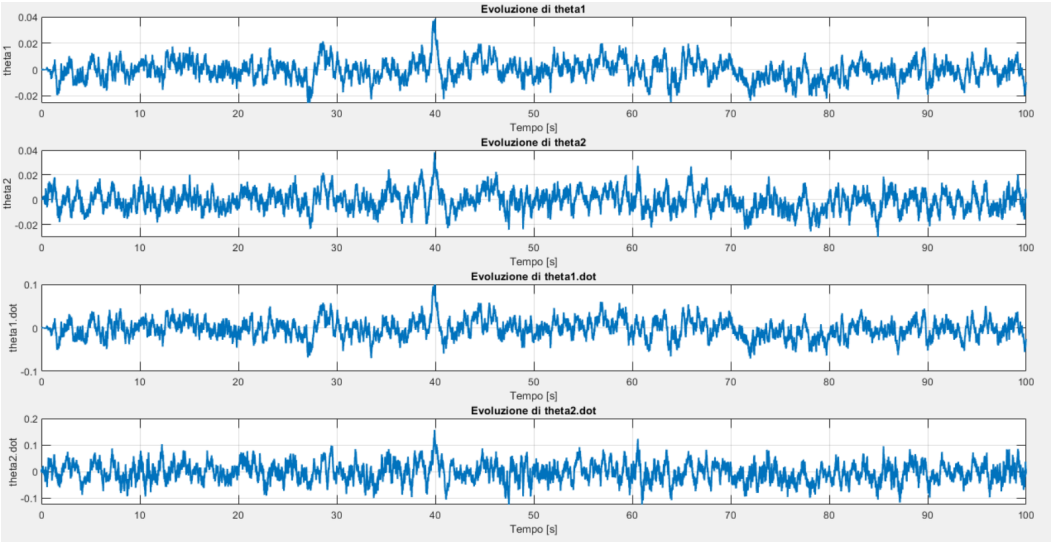


Figure 10: Differenza stati reali modello e stati stimati UKF

Da questo grafico si evince la bontà della stima UKF che dimostra di stimare molto bene gli angoli (errore max 0,04) e discretamente bene le velocità angolari (errore max 0,2) per motivi sopra citati. In tutti gli stati la media è approssimabile a zero quindi ci aspettiamo una traiettoria dell'end-effector stimata che oscilla intorno alla traiettoria reale con piccolo errore.

- **Innovazione UKF**

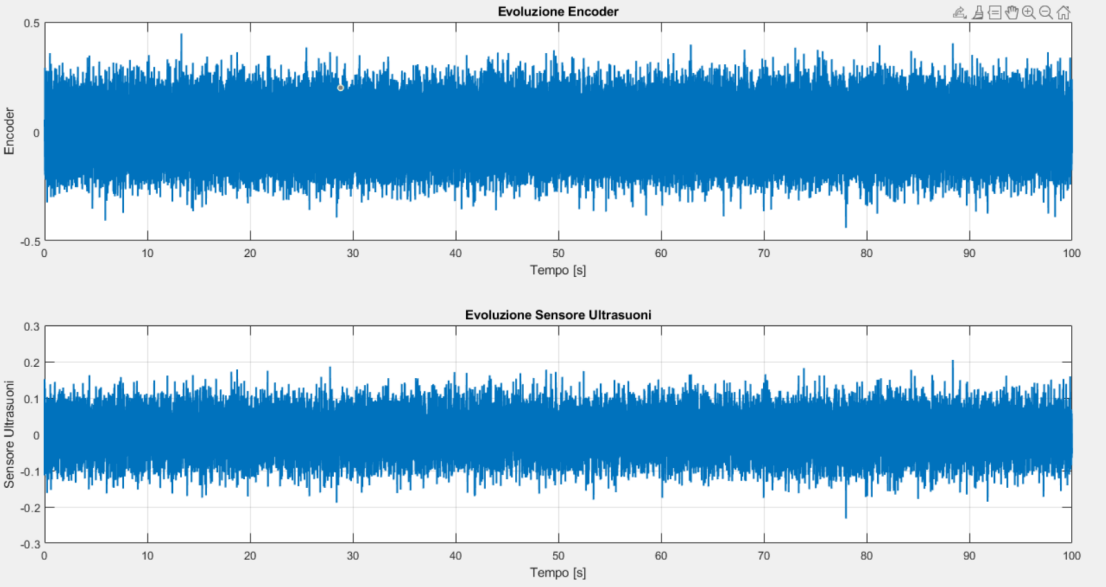


Figure 11: Innovazione dei due sensori con stima UKF

L'innovazione ricavata dalla differenza delle misure dei sensori e dalle stime ad ogni passo, ha la capacità di indicarci quanto la stima sia in accordo con il modello reale, controllando che sia un rumore bianco. Qualitativamente entrambi i grafici sembrano essere rumore bianco, ma per sicurezza abbiamo verificato applicando ai vettori e_k la **funzione lbqtest** che controlla le correlazioni tra tutti i punti (se h=0 e $p > 0.05$ è rumore bianco).
Si fa presente che per graficare e_k è stata esclusa la generazione degli outlier che avrebbero inevitabilmente sporcato il segnale essendo direttamente funzione delle misure.

	h	p
ek(1)	0	0.53
ek(2)	1	0

Table 1: Risultati funzione lbqtest(eUKF)

I risultati del controllo (Tabella1) per $e_k(1)$ sono coerenti ad un rumore bianco, diversamente per $e_k(2)$Ṗuesto perchè il secondo sensore, ovvero quello a ultrasuoni, lavora a una frequenza ridotta rispetto al sistema e alla stima (misura ogni 0.003s), questo comporta che nella differanza misura-stima che viene fatta ogni 0.001s la misura rimanga indietro per qualche passo rispetto alla stima che comunque propagherà entrambi gli stati ad ogni passo. A conferma di ciò abbimo effettuato una prova allineando le frequenze dei sensori con il sistema e i risultati indicano per entrambi i segnali rumore bianco, dimostrando anche che la stima sia effettivamente concorde al modello.

- **Variazione dei parametri rispetto ai valori reali**

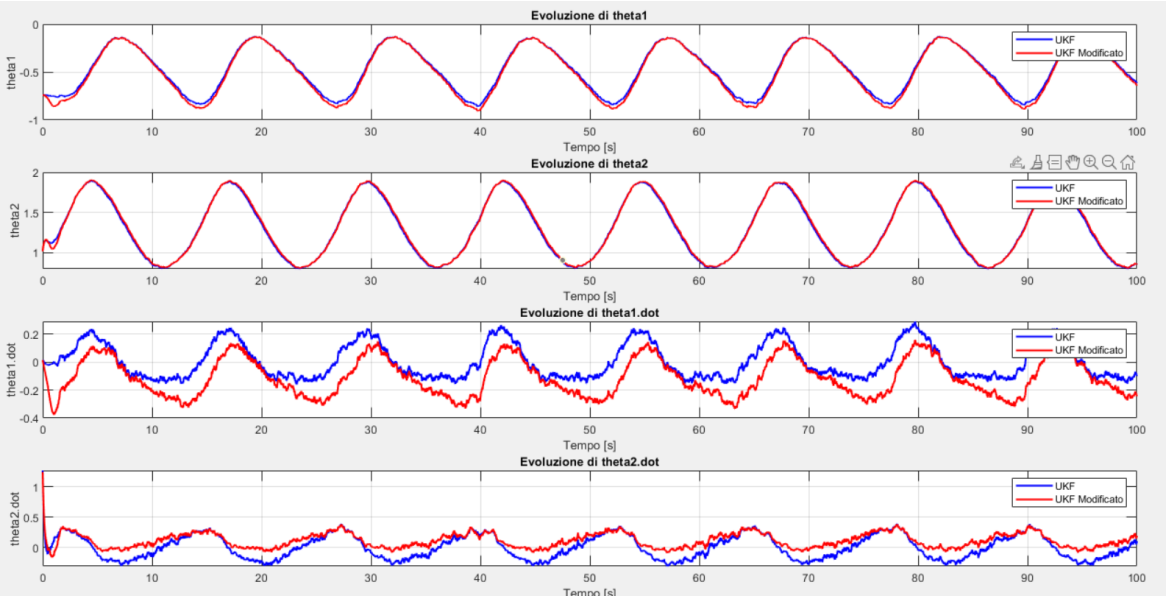


Figure 12: Confronto stima UKF(blu) e stima UKF con variazione di alcuni parametri(rosso)

Per concludere l'analisi della stima UKF abbiamo provato a cambiare alcuni parametri del modello inserito nella stima rispetto a quelli reali del braccio:

- M1 = M1 + 1 kg
- M2 = M2 + 0.5 kg
- c1 = c1 + 0.05
- c2 = c2 + 0.05

In generale si può dire che la stima sia ancora considerabile buona, ma come si può notare la vaiazione maggiore si ha nelle velocità angolari che spostano il loro valor medio verso il basso. Questo ci indica che la stima segurà il movimento dell'end-effector reale con un'piccolo offset aggiuntivo rispetto alla stima UKF normale. Per capire se rimanesse effettivamente una stima valida abbiamo ricavato anche in questo caso l'innovazione mettendo le frequesnze dei sensori entrambe in accordo con il modello (per un controllo oggettivo solo della stima) e applicata lbqtest :

	h	p
ek(1)	0	0.40
ek(2)	0	0.28

Table 2: Risultati funzione lbqtest(e_{UKF}) con parametri modificati e frequenze sensori conguenti

Come si evince dei risultati (Tabella2) anche variando alcuni parametri la stima rimane in accordo con il modello reale.

4 EKF

L'implementazione dell'EKF (Extended Kalman Filter) si sviluppa attraverso due fasi suddivise in due differenti blocchi simulink, la fase di predizione e quella di correzione.

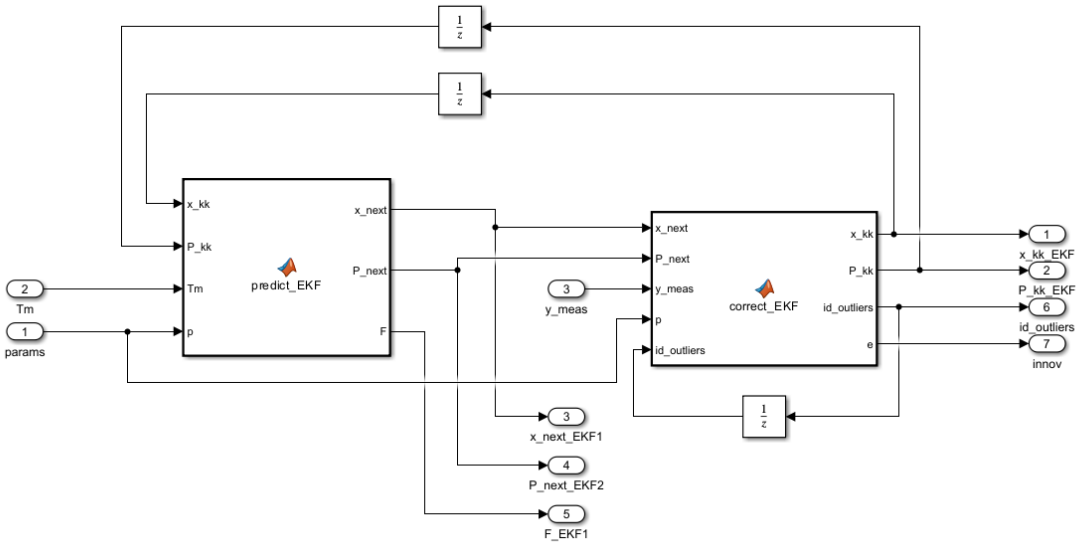


Figure 13: Blocco simulink EKF.

4.1 Predizione

In figura 13 notiamo che gli ingressi nel blocco di predizione sono i parametri, le coppie ai giunti τ_m che derivano dal blocco di traiettoria desiderata, la covarianza $P_{k|k}$ e la stima $x_{k|k}$ (uscite dal blocco di correzione e retroazionate con un blocco di unit delay). Nei parametri viene definita anche la varianza del rumore σ_W che servirà successivamente per costruire la matrice Q mentre i disturbi τ_{d1} e τ_{d2} sono impostati nulli.
Le uscite del blocco di predizione sono la covarianza P_{next} e la stima x_{next} . In figura 14 viene riportato il calcolo della stima predetta (x_{next}) che avviene applicando il modello e tenendo in considerazione i disturbi nulli, mentre per effettuare il calcolo di P_{next} si prelevano le matrici F e D da uno script matlab che utilizza la funzione jacobian per effettuare le derivate parziali.

```
% Aggiornamento degli stati con Eulero
x_next(1) = theta1 + theta1_dot * dt;
x_next(2) = theta2 + theta2_dot * dt;
x_next(3) = theta1_dot + theta_ddot(1) * dt;
x_next(4) = theta2_dot + theta_ddot(2) * dt;

% Covarianza Errore Predizione
Q = [sigma_W^2, 0; 0, sigma_W^2];
Q = T * Q * T';
P_next = F * P_kk * F' + D * Q * D';
```

Figure 14: Blocco di predizione.

4.2 Correzione

Le uscite della predizione diventano gli ingressi del blocco di correzione insieme ai parametri e alle misure dei sensori. Nei parametri si inseriscono le varianze dei rumori nelle misure di encoder e

sensore ad ultrasuoni (σ_{v1}, σ_{v2}) per poter costruire la matrice R.
Anche in questo caso per calcolare le matrici H e M si utilizza lo script di Matlab ma una volta riportate su simulink il calcolo di H si divide in due diverse casistiche che dipendono dal segno dell'angolo del primo giunto.
Come riportato in figura 15, si calcolano l'innovazione, la covarianza dell'innovazione, il guadagno di correzione e si implementa la distanza di Mahalanobis con un'opportuna soglia fissata a 4 così da individuare eventuali outliers.
Se la misura su cui ci si basa è identificata come outlier allora la stima e la covarianza rimaranno uguali a quelle predette mentre se è una misura valida si ricavano grazie alle grandezze trovate precedentemente la stima corretta $x_{k|k}$ e la covarianza $P_{k|k}$ utilizzando la formula di Joseph per scongiurare eventuali errori e perdere di positività.

```
e = y_meas - y_cor;
e(1,1) = atan2(sin(e(1,1)),cos(e(1,1)));
S_k = H * P_next * H' + Mh * R * Mh';
L_k = P_next * H' / (S_k);
mahalanobis_distance = sqrt(e' / (S_k) * e);

if mahalanobis_distance>mahalanobis_threshold && MAHALANOBIS
      x_kk = x_next;
      P_kk = P_next;
      id_outliers = id_outliers + 1;
else
      x_kk = x_next + L_k * (e);
      P_kk = (eye(4) - L_k * H) * P_next * (eye(4) - L_k * H)' + L_k * Mh * (R) * Mh' * L_k';
end
```

Figure 15: Blocco di correzione

5 Risultati EKF

5.1 Confronto EKF e modello reale

Di seguito in figura 16 vediamo l'andamento dello stato stimato con EKF al passare del tempo messo a confronto con lo stato reale del sistema. Notiamo come i primi due stati che controllano le variabili di giunto e quindi le posizioni dell'end effector sono simili mentre si discostano più dalla realtà le velocità angolari con cui si muovono i link. Questo avviene per lo stesso motivo descritto precedentemente nell'analisi del filtro UKF, poichè facciamo uso dello stesso modello di propagazione.

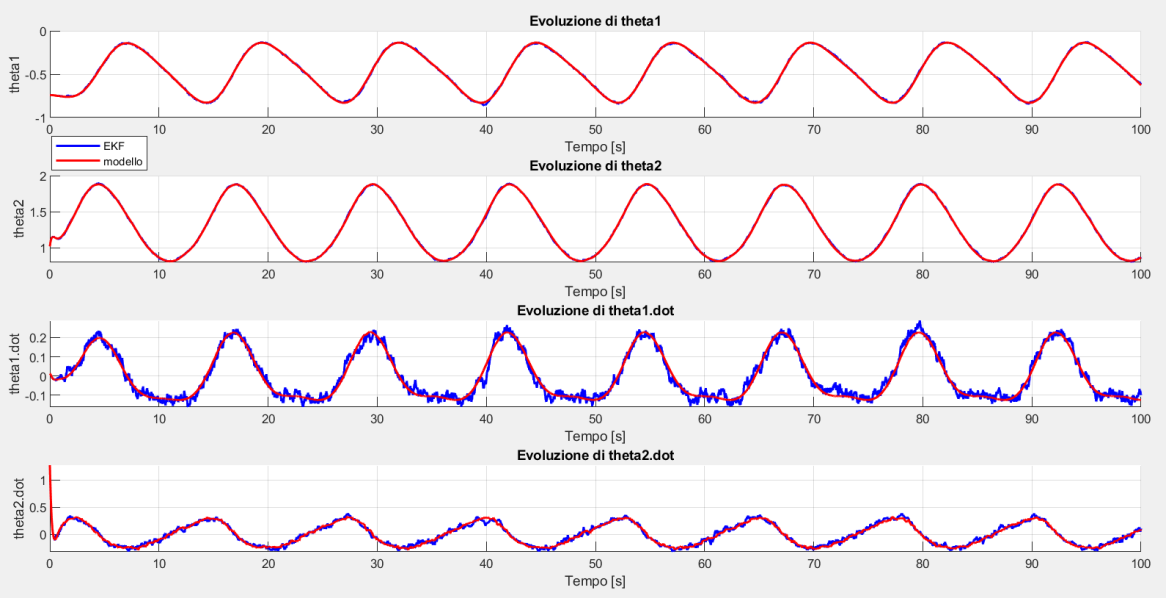


Figure 16: Confronto tra EKF e modello reale.

In figura 17, è stato riportato l'errore tra EKF e modello, questo grafico conferma ciò che abbiamo detto in precedenza, per le prime due variabili di stato l'errore rimane sull'ordine di grandezza di 10^{-2} mentre per le velocità angolari l'errore aumenta di un ordine di grandezza.

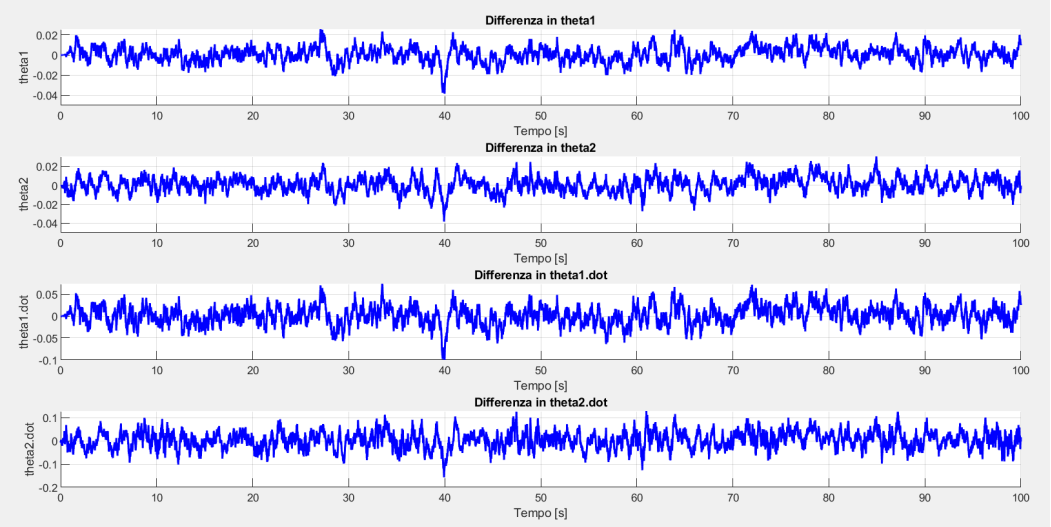


Figure 17: Differenza tra EKF e modello.

5.2 Innovazione

Per poter verificare che ci sia accordo tra modelli e sistema reale abbiamo graficato l'innovazione data dall'encoder e quella data dal sensore ad ultrasuoni.

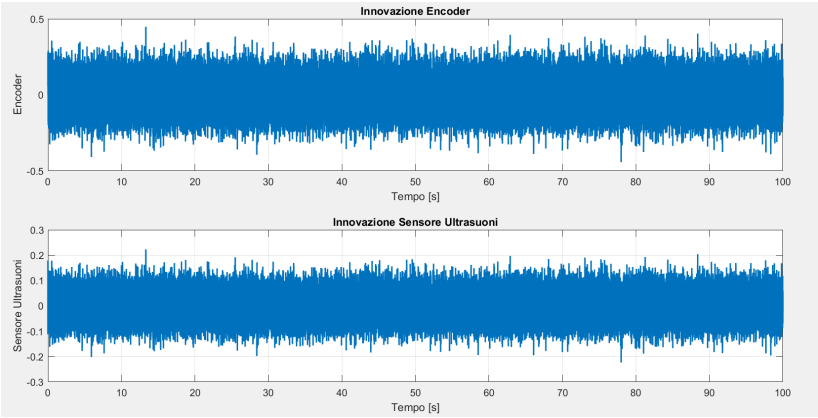


Figure 18: Innovazione EKF.

Il grafico in figura 18 riporta entrambe le innovazioni calcolate con le misure dei due sensori. Per poter verificare la bianchezza delle innovazioni si utilizza uno scipt su matlab in cui la funzione lbqtest fornisce due grandezze h e p sopracitate.

Sensore	h	p
Encoder	0	0.4062
Ultrasuoni	1	0

Table 3: Tabella di confronto tra innovazione e rumore bianco.

La tabella 3 ci dimostra che per l'encoder l'innovazione è un rumore bianco mentre per il sensore ad ultrasuoni risulta non esserlo come abbiamo verificato anche nel caso del filtro UKF, a causa delle diverse frequenze.
Provando a cambiare le frequenze dei sensori imponendole entrambe uguali (misure ogni 0.001s)

notiamo dalla tabella 4 come anche in questo caso i valori e i risultati cambino esattamente come nel filtro UKF in cui entrambe le innovazioni diventano rumori bianchi.

Sensore	h	p
Encoder	0	0.3479
Ultrasuoni	0	0.7107

Table 4: Tabella di confronto tra innovazione e rumore bianco con i sensori alla stessa frequenza.

5.3 Analisi di robustezza

Per poter verificare la robustezza della stima abbiamo inserito nel blocco di predizione delle variazioni nei parametri, in particolare sulle masse dei due link e sui loro coefficienti di attrito viscoso. Le variazioni aggiunte ai parametri sono le stesse inserite nell'UKF per riuscire a fare un confronto diretto. In figura 18 vediamo come rispetto al modello i primi due stati che controllano le variabili di giunto e quindi le posizioni dell'end effector sono simili mentre si discostano più dalla realtà le velocità angolari con cui si muovono i link.

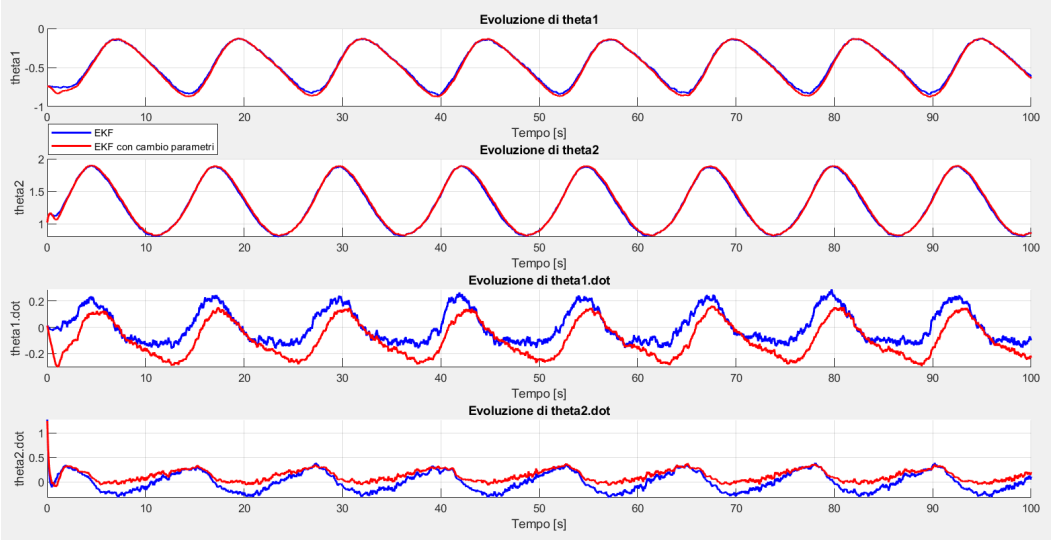


Figure 19: Differenza tra EKF ed EKF con variazione di parametri.

Calcolando invece l'innovazione nel caso dei parametri variati (sempre utilizzando la stessa frequenza nei sensori) notiamo in tabella 5 come a differenza del filtro UKF l'innovazione legata al secondo sensore non sia un rumore bianco. Questo porterebbe alla conclusione che il filtro EKF sia un po' meno robusto alle variazioni rispetto all'UKF nel caso in esame.

Sensore	h	p
Encoder	0	0.8130
Ultrasuoni	1	0

Table 5: Tabella di confronto tra innovazione e rumore bianco con i parametri variati e i sensori alla stessa frequenza.

6 RTS

6.1 Implementazione

La parte di regolarizzazione viene realizzata attraverso uno script matlab in cui vengono calcolate la stima regolarizzata e la covarianza ad essa associata, utilizzando per il calcolo la matrice C_k che fa riferimento alla matrice F_k estratta dalla predizione dell'EKF assieme al $x_{k+1|k}$ $P_{k+1|k}$ e a $x_{k|k}$ e $P_{k|k}$ invece derivate dalla parte di correzione.

```
for k = size(x_kk,1)-1:-1:1

    %Ck = P_k|k * F_k+1' * P_k+1|k^(-1)
    Ck = P_kk(:,:,k) * F_k(:,:,k+1)' / P_next(:,:,k+1);

    %x_k|n = x_k|k + Ck*(x_k+1|n - x_k+1|k)
    x_smoothed(k,:) = x_kk(k,:)' + Ck*(x_smoothed(k+1,:)' - x_next(:,:,k+1));

    %P_k|n = P_k|k + Ck*(P_k+1|n - P_k+1|k)*Ck'
    P_smoothed(k,:,:) = P_kk(:,:,k) + Ck*(P_smoothed(k+1,:,1) - P_next(:,:,k+1))*Ck';
end
```

Figure 20: Codice RTS.

6.2 Risultati RTS

Per valutare i risultati ottenuti ci basiamo sulla figura 21 che ci mostra il confronto tra EKF regolarizzato ed EKF. Il grafico ci dà l'idea di come la regolarizzazione influisca e migliori la stima soprattutto nelle variabili di stato che fanno riferimento alle velocità angolari dei links. Questo processo permette di smussare il loro comportamento grazie alle misure e ai calcoli che si effettuano su dati futuri rispetto all'istante stimato.

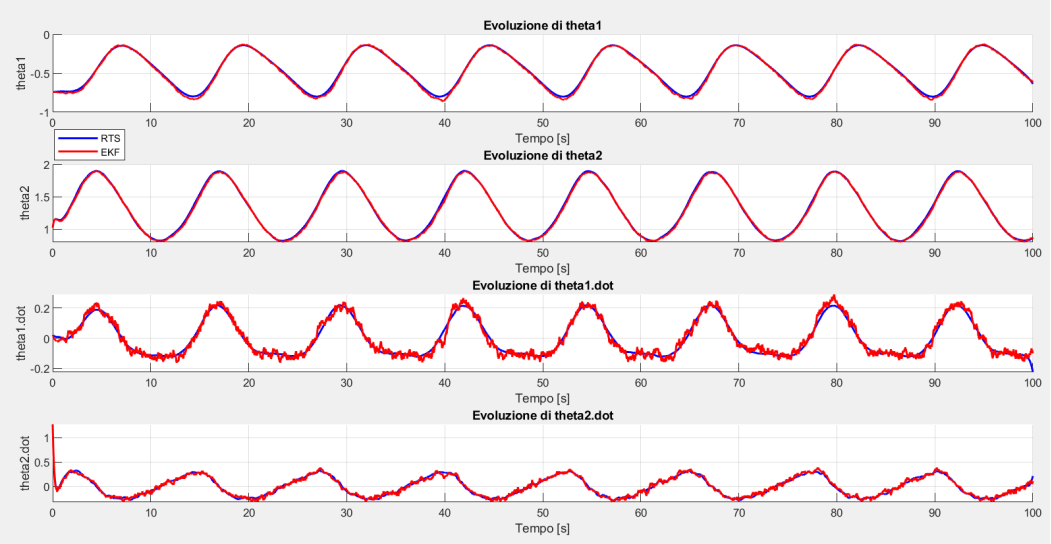


Figure 21: Confronto tra EKF e RTS.

7 Conclusioni e Confronti

Dopo aver analizzato le due stime possiamo concludere che sia UKF che EKF effettuano una buona stima del braccio robotico, con comportamenti molto simili quasi sovrapponibili.

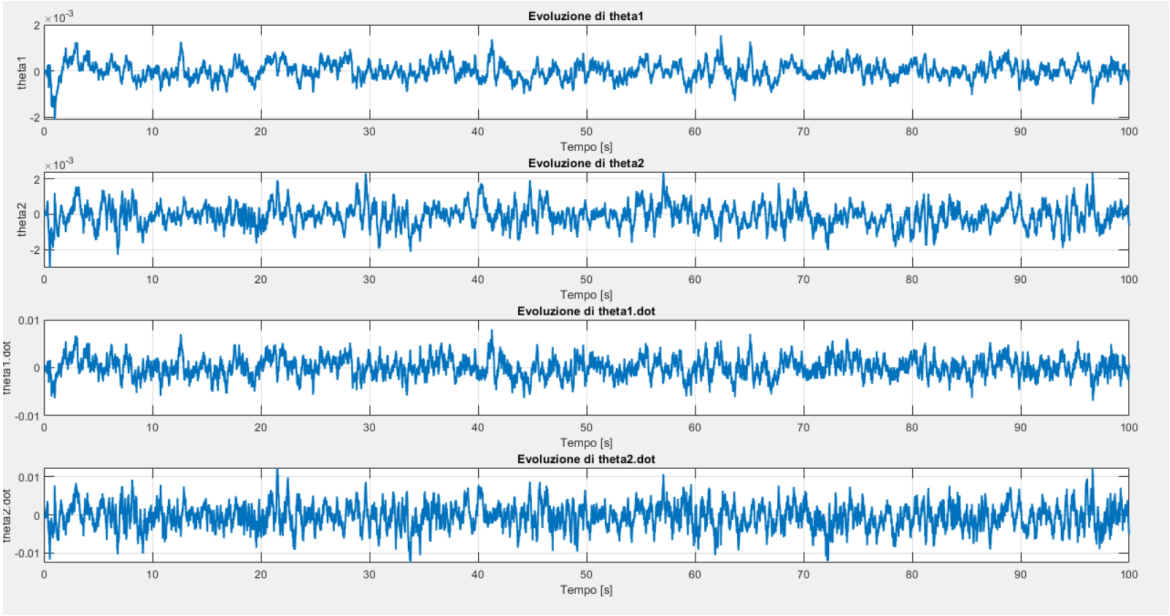


Figure 22: Differenza tra UKF e EKF

Per rappresentare visivamente il risultato delle stime abbiamo creato un'animazione che mostra il movimento del braccio robotico reale e le posizioni stimate dell'end-effector (Figura 23).

Come ci aspettavamo le due traiettorie stimate sono pressocchè sovrapposte e entrambe stimano discretamente la traiettoria reale, discostandosi poco da essa (in Figura 23 la finestra è molto ingrandita).
Nell'applicazione pratica EKF risulta sicuramente una tecnica di stima più complessa da applicare a causa del calcolo delle derivate. Se inoltre aggiungiamo che nel caso di errori di modello, che abbiamo simulato con delle variazioni dei parametri, la stima EKF risulta meno robusta rispetto all'UKF in questo caso potrebbe essere preferibile utilizzare la stima UKF.
Per quanto riguarda la regolarizzazione con RTS ci fornisce un risultato migliore rispetto a tutte le altre stime come era ragionevole aspettarsi. Quindi se non ho limitazioni date dal tempo e la potanza di calcolo sicuramente sarebbe una valida opzione quella di effettuare EKF+RTS come ci conferma la figura 24 che mette in risalto quanto sia migliore la traiettoria regolarizzata rispetto all'EKF e quindi anche all'UKF.

Nell'ultima Figura 25 possiamo notare che se ripetiamo l'animazione variando i parametri delle stime entrambe in egual modo, si nota un maggior discostamento dalla traiettoria reale, quindi un peggioramento della stima. Inoltre ingrandendo si osserva che la traiettoria dell'EKF è sempre leggermente più lontana da quella reale rispetto a quella dell'UKF. Questo è in accordo con i risultati estratti dall' analisi dell'innovazione che evidenziavano una sensibilità maggiore agli errori di modello dell'EKF.

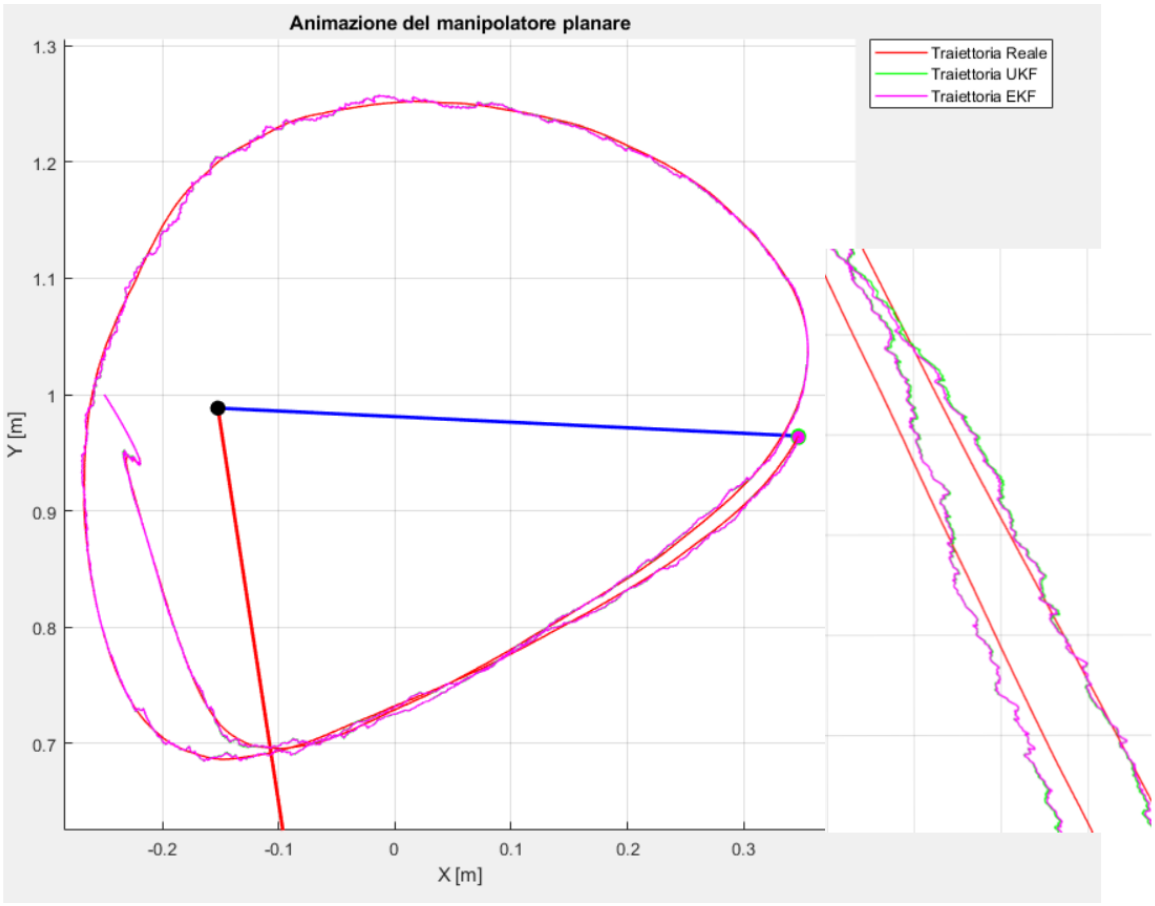


Figure 23: Braccio robotico con traiettorie stimate

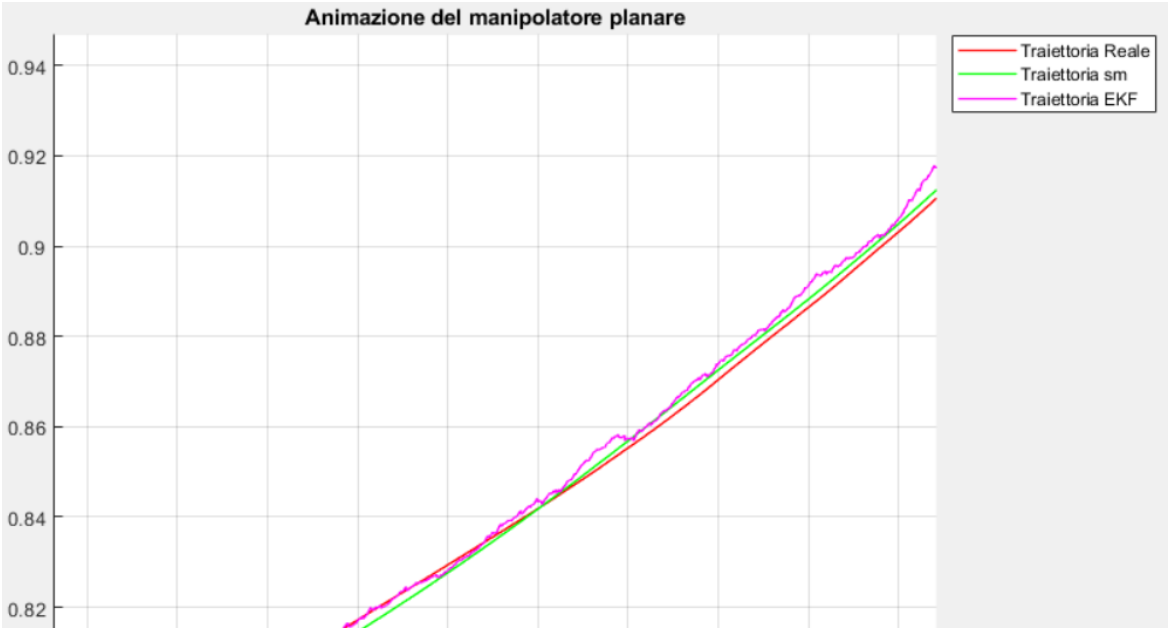


Figure 24: Braccio robotico con traiettorie EKF e EKF+RTS

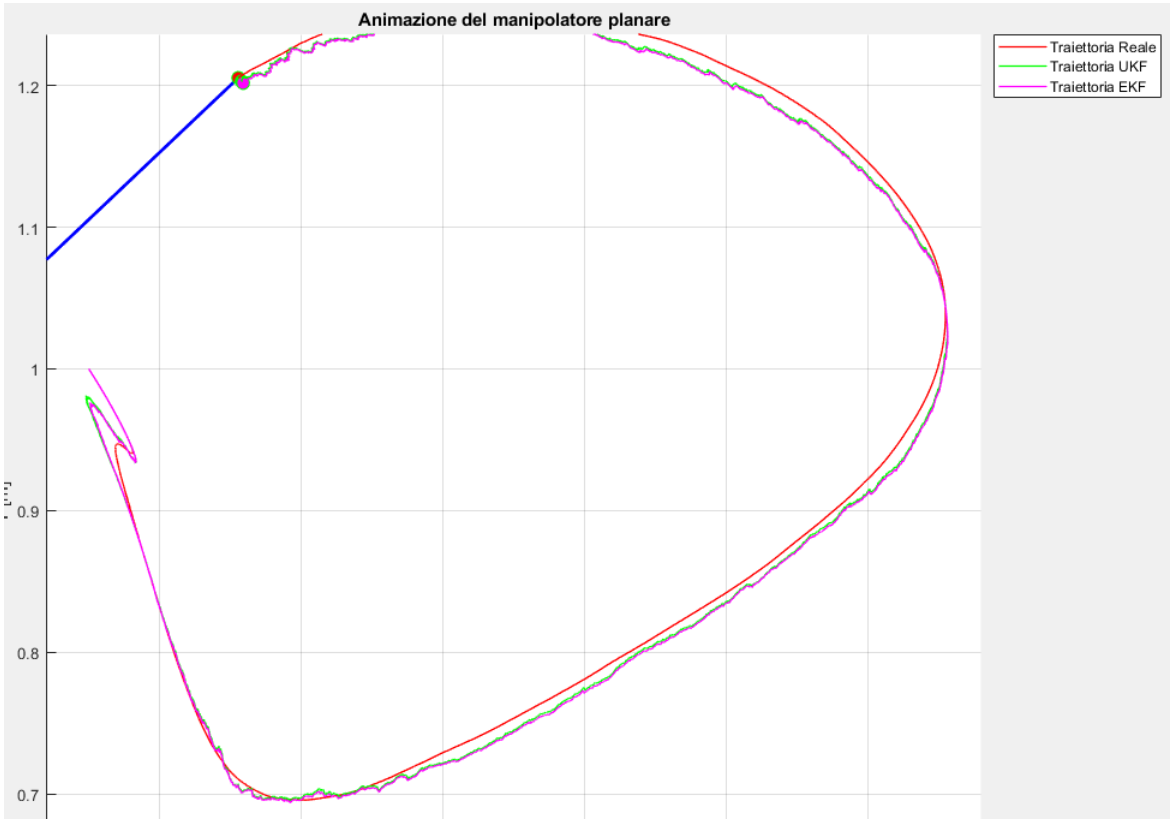


Figure 25: Braccio robotico con traiettorie stimate con parametri variati